Object Oriented Analysis and Design

SHENKAR שנקר
ENGINEERING. DESIGN. ART. הנדסה. עיצוב. אמנות.

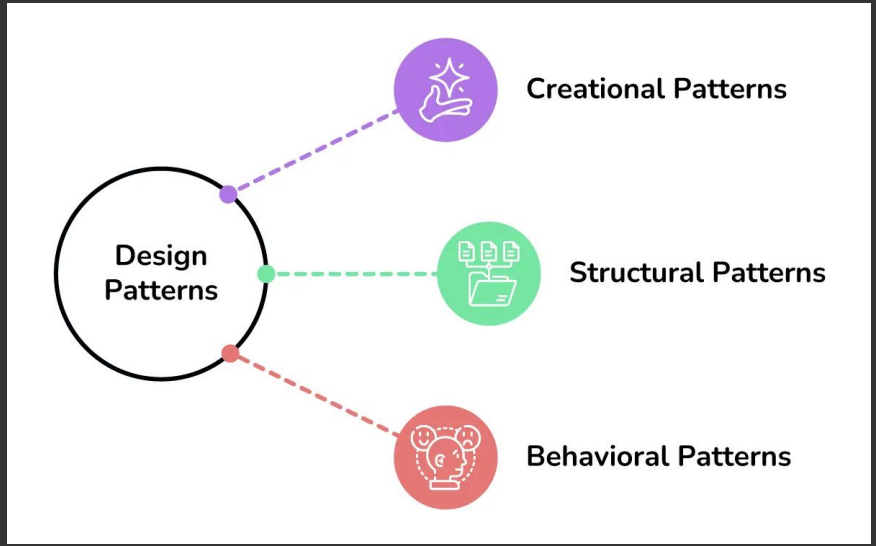
Design Patterns:
Template Method , Strategy, Iterator

Vladimir Soroka

3 Categories Of Patterns

SHENKAR — שנקר
ENGINEERING. DESIGN. ART. הנדסה. עיצוב. אמנות

- **Creational** – object creation
- **Structural** – class/object composition
- **Behavioral** – object interaction & responsibility



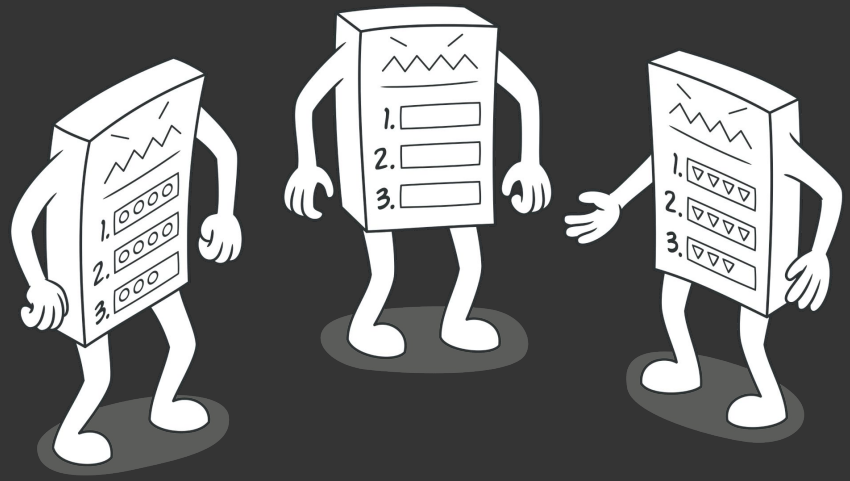
What are we covering?

SHENKAR שנקר
ENGINEERING. DESIGN. ART. הנדסה. עיצוב. אמנות

- **Template Method** – behavioral
- **Strategy -** behavioral
- **Iterator** - behavioral

The Template Method Pattern

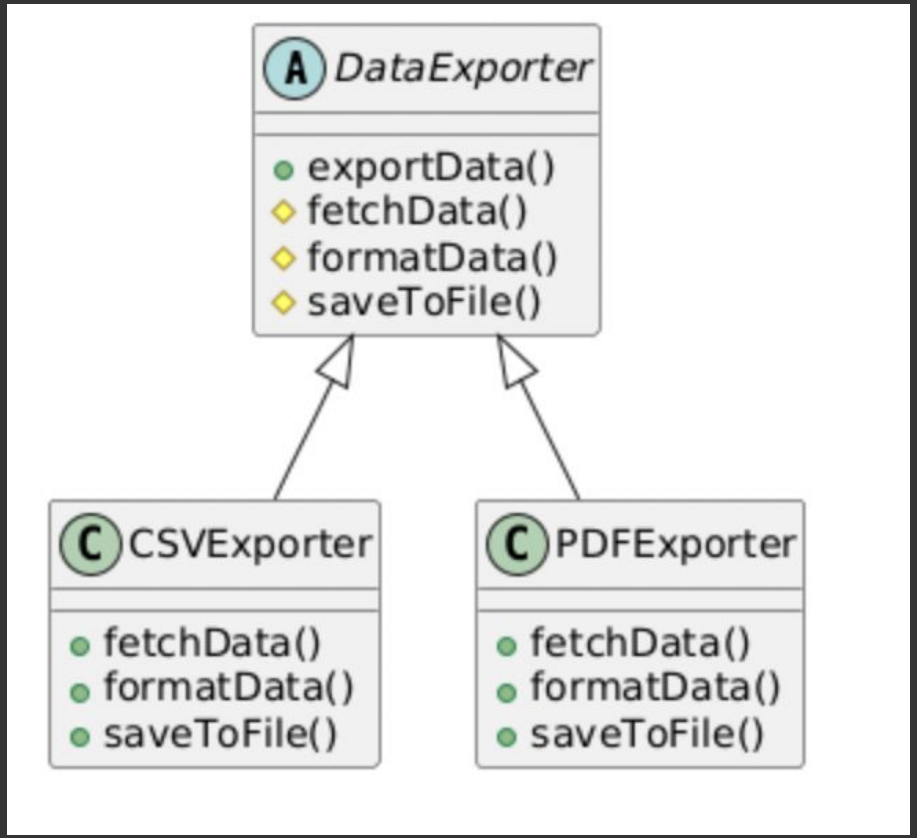
- **Purpose**
  - Defines the **skeleton of an algorithm** in a method, deferring some steps to subclasses.
  - Lets subclasses **redefine specific steps** without changing the algorithm structure.
  - Promotes **code reuse** and **behavior consistency** across similar classes.



📌 **Analogy**: “Think of a recipe template – the steps are fixed, but the ingredients can vary.”

Template Method UML

```
@startuml
abstract class DataExporter {
  + exportData()
  # fetchData()
  # formatData()
  # saveToFile()
}

class CSVExporter {
  + fetchData()
  + formatData()
  + saveToFile()
}

class PDFExporter {
  + fetchData()
  + formatData()
  + saveToFile()
}

DataExporter <|-- CSVExporter
DataExporter <|-- PDFExporter
@enduml
```



Template Method - Example

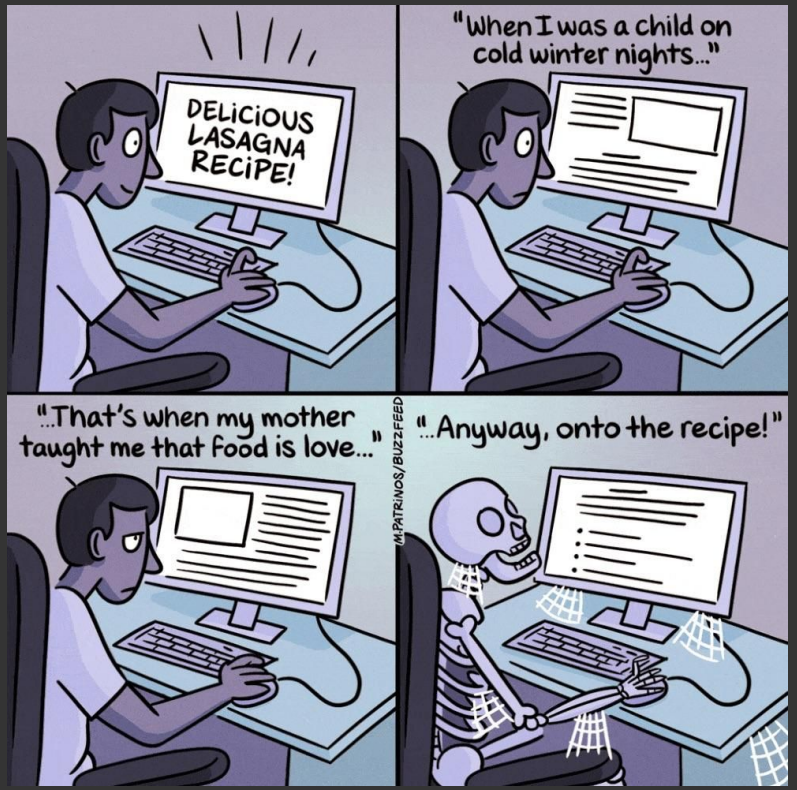

```
class DataExporter {
public:
    void exportData() {
        fetchData();
        formatData();
        saveToFile();
    }

protected:
    virtual void fetchData() = 0;
    virtual void formatData() = 0;
    virtual void saveToFile() {
        std::cout << "Saving to default path..." << std::endl;
    }
};
```

Template Method - Example


```
class CSVExporter : public DataExporter {
protected:
    void fetchData() override {
        std::cout << "Fetching data for CSV..." << std::endl;
    }
    void formatData() override {
        std::cout << "Formatting data as CSV." << std::endl;
    }
};

class PDFExporter : public DataExporter {
protected:
    void fetchData() override {
        std::cout << "Fetching data for PDF..." << std::endl;
    }
    void formatData() override {
        std::cout << "Formatting data as PDF." << std::endl;
    }
    void saveToFile() override {
        std::cout << "Saving PDF to secure folder." << std::endl;
    }
};
```

When to Use Template Method?

- You have multiple operations that follow **identical structure**, with varying steps.
- You want to **prevent duplicate logic** in subclasses.
- You want to **enforce an execution order**.



What’s Bad About Template Method?

- **Inheritance Lock-In** – Can't change behavior without subclassing.
- **Class Explosion** – Every variation requires a new subclass.
- **Hidden Flow** – Template method logic split between base and subclass.
- **Harder Testing** – Need to instantiate hierarchy just to test flow.

Template Method Summary

- Template Method lets you fix **workflow structure** while customizing parts.
- Keeps shared logic in the base class, variation in subclasses.
- Use when structure must remain **stable**, but steps can change.
- Be cautious of **rigid hierarchies** and consider alternatives like **Strategy**.

The Strategy Pattern

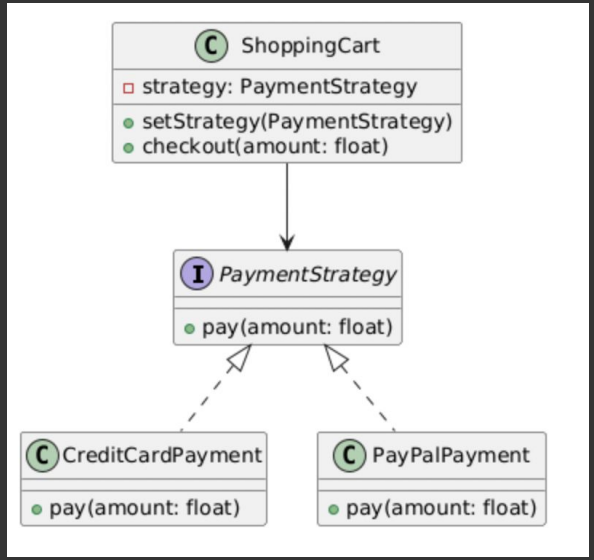
Purpose

- Encapsulates **interchangeable algorithms** or behaviors.
- Allows selecting an algorithm **at runtime**.
- Promotes **open/closed principle**: new strategies without modifying existing code.

📌 **Analogy**: “Like choosing between driving, biking, or walking to work – same goal, different strategies.”

Strategy UML

```
@startuml

interface PaymentStrategy {

  + pay(amount: float)

}


class CreditCardPayment {

  + pay(amount: float)

}


class PayPalPayment {

  + pay(amount: float)

}


class ShoppingCart {

  - strategy: PaymentStrategy

  + setStrategy(PaymentStrategy)

  + checkout(amount: float)

}


PaymentStrategy <|.. CreditCardPayment

PaymentStrategy <|.. PayPalPayment

ShoppingCart --> PaymentStrategy

@enduml
```



Strategy Code - Example

SHENKAR — שנקר
ENGINEERING. DESIGN. ART. הנדסה. עיצוב. אמנות.

```cpp
class PaymentStrategy {
public:
    virtual void pay(float amount) = 0;
    virtual ~PaymentStrategy() = default;
};

class ShoppingCart {
    PaymentStrategy* strategy;
public:
    void setStrategy(PaymentStrategy* s) {
        strategy = s;
    }

    void checkout(float amount) {
        if (strategy)
            strategy->pay(amount);
        else
            std::cerr << "No payment strategy selected.\n";
    }
};
```

Strategy Code - Example


```
class CreditCard : public PaymentStrategy {
public:
    void pay(float amount) override {
        std::cout << "Paid $" << amount << " using credit card.\n";
    }
};

class PayPal : public PaymentStrategy {
public:
    void pay(float amount) override {
        std::cout << "Paid $" << amount << " using PayPal.\n";
    }
};
```

Strategy Code - Example


```
int main() {
    ShoppingCart cart;

    CreditCard card;
    cart.setStrategy(&card);
    cart.checkout(100.0);

    PayPal paypal;
    cart.setStrategy(&paypal);
    cart.checkout(200.0);
}
```

```
Paid $100 using credit card.
Paid $200 using PayPal.
```

When to Use Strategy?

- You have many **variations of the same algorithm**.
- You want to **select behavior at runtime**.
- You want to **eliminate long conditionals** (e.g., if...else or switch-case).
- You want to follow **composition over inheritance**.

What’s Bad About Strategy?

SHENKAR — שנקר
ENGINEERING. DESIGN. ART. הנדסה. עיצוב. אמנות

1. Strategy Selection is Manual

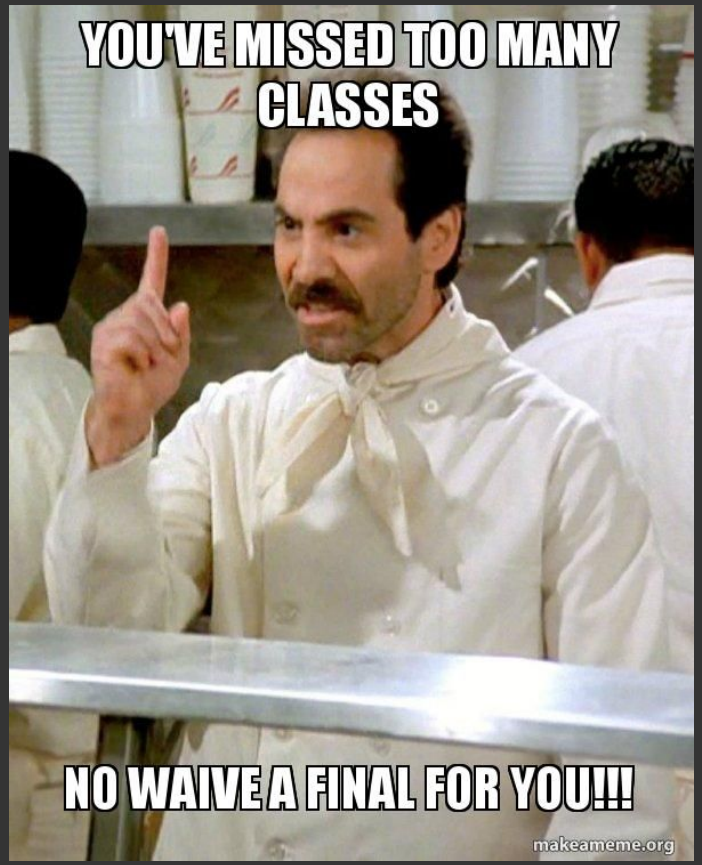
```
if (userChoosesCard)
    cart.setStrategy(&card);  // Client must know strategies
```

🧠 Requires external logic or UI input to pick a strategy.

2. Too Many Classes

```
SortAscending, SortDescending, SortRandomized, ...
```

😱 Explodes in complexity if each variation is implemented separately.



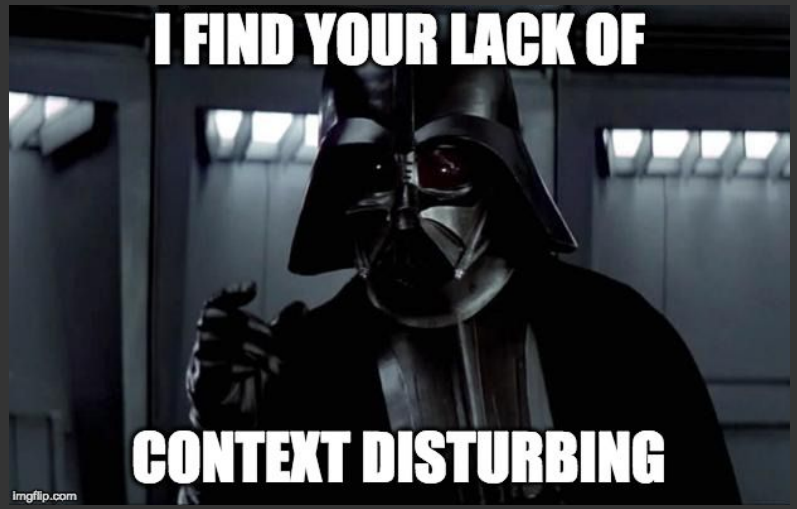
What’s Bad About Strategy?

SHENKAR — שנקר
ENGINEERING. DESIGN. ART. הנדסה. עיצוב. אמנות.

3. Strategies May Need Context Info

```
// Strategy can't access customer profile unless passed manually

strategy->pay(user.amount);  // limited context
```

💔 Breaks encapsulation if not designed well.



4. Performance Overhead (if misused)

- Too much indirection in performance-critical paths.
- Simple strategy could be inlined.

Strategy - Summary

- Strategy lets you **encapsulate algorithms** and swap them at runtime.
- Promotes clean code and avoids giant conditional blocks.
- Ideal for **UI behaviors, AI logic, business rules**.
- Use with care: balance flexibility vs complexity.

The Iterator Pattern

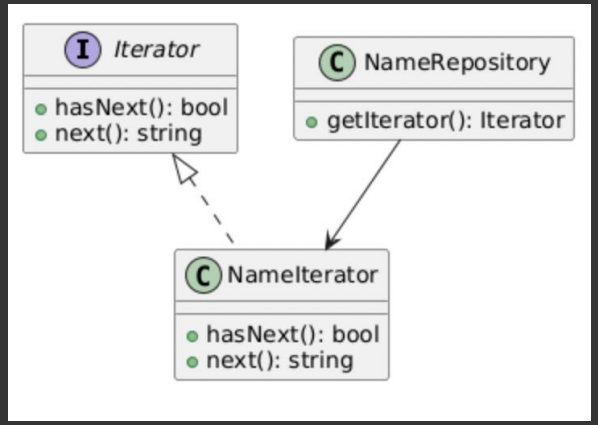
Purpose

- Provides a **standard way to access elements** of a collection sequentially.
- Allows traversal without exposing the collection's **internal structure**.
- Enables **multiple concurrent traversals**.

📌 **Analogy**: "Like using a remote control to browse TV channels without opening the TV casing."

Iterator UML

```
@startuml
interface Iterator {
  + hasNext(): bool
  + next(): string
}

class NameIterator {
  + hasNext(): bool
  + next(): string
}

class NameRepository {
  + getIterator(): Iterator
}

Iterator <|.. NameIterator
NameRepository --> NameIterator
@enduml
```



Iterator Code - Example


```cpp
template<typename T>
class Iterator {
public:
    virtual bool hasNext() = 0;
    virtual T next() = 0;
    virtual ~Iterator() = default;
};

class NameRepository {
    std::vector<std::string> names;
public:
    NameRepository() : names{"Alice", "Bob", "Charlie"} {}

    class NameIterator : public Iterator<std::string> {
        const std::vector<std::string>& names;
        size_t index = 0;
    public:
        NameIterator(const std::vector<std::string>& n) : names(n) {}
        bool hasNext() override { return index < names.size(); }
        std::string next() override { return names[index++]; }
    };

    std::unique_ptr<Iterator<std::string>> getIterator() {
        return std::make_unique<NameIterator>(names);
    }
};
```

Strategy Code - Example


```
int main() {
    NameRepository repo;
    auto it = repo.getIterator();

    while (it->hasNext()) {
        std::cout << it->next() << std::endl;
    }
}
```

When to Use Iterator?

- You want to **hide the internal structure** of a collection.
- You need to **traverse complex data** (trees, graphs) in a consistent way.
- You want to **support multiple simultaneous traversals** (e.g., cursors).
- You want a **uniform interface** for different types of collections.

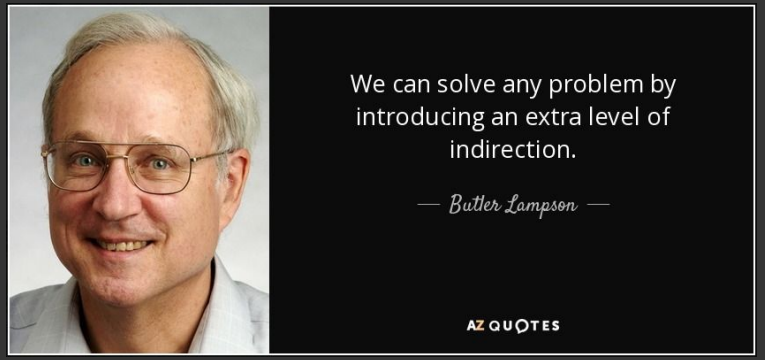
What’s Bad About Iterator?

SHENKAR שנקר
ENGINEERING. DESIGN. ART. הנדסה. עיצוב. אמנות

1. Extra Indirection

```
std::vector<int>::iterator it =
vec.begin();  // More verbose than
range-for
```

🔁 Adds boilerplate over native language features in modern C++.



2. External Iterators Require Manual Management

```
while (it->hasNext()) { ... }  //
You must control the loop manually
```

📉 Easy to misuse or forget termination conditions.

What’s Bad About Iterator?

3. Can Expose Internal State (Poor Encapsulation)

If your iterator **returns pointers** or exposes internal nodes:

- `Node* next();  // Allows modification of internal state`

⚠️ Breaks encapsulation and leads to tight coupling.

4. Can Be Hard to Optimize

- For large or concurrent structures, simple iterators are **not efficient**.
- May not support **parallel access**, filtering, or lazy evaluation.

Iterator - Summary

- Iterator abstracts collection traversal.
- Decouples data structure from traversal logic.
- Ideal for **tree, graph, or composite** data structures.
- External iterators are flexible but require care.
- Modern C++ offers alternatives: **range-for**, **ranges**, **STL iterators**.

Bad Examples

Example 1


```
class Exporter {
public:
    void exportData() {
        open();
        write();
        close();
    }
    virtual void open() = 0;
    virtual void write() = 0;
    virtual void close() = 0;
};

class CSVExporter : public Exporter {
    void open() override { ... }
    void write() override { ... }
    void close() override { ... }
};
```

Example 2


```
class Base {
public:
    virtual void algorithm() {
        step1();
        step2();
    }
    virtual void step1() {}
    virtual void step2() {}
};

class Sub : public Base {
public:
    void algorithm() override { std::cout << "I'll do it my way"; }
};
```

Example 3


```
void process() {
    auth();
    log();
    validate();
    transform();
    compress();
    store();
    notify();
    cleanup();
    audit();
}
```

Example 4


```
if (type == "PayPal")
    cart.setStrategy(new PayPal());
else if (type == "Card")
    cart.setStrategy(new CreditCard());
```

Example 5


```
class FastSort : public SortStrategy {
public:
    void sort() {
        logStart();
        quickSort();
        logEnd();
    }
};

class SlowSort : public SortStrategy {
public:
    void sort() {
        logStart();
        bubbleSort();
        logEnd();
    }
};
```

Example 6


```
class Context {
    SortStrategy* strategy = new QuickSort();
};
```

Example 7

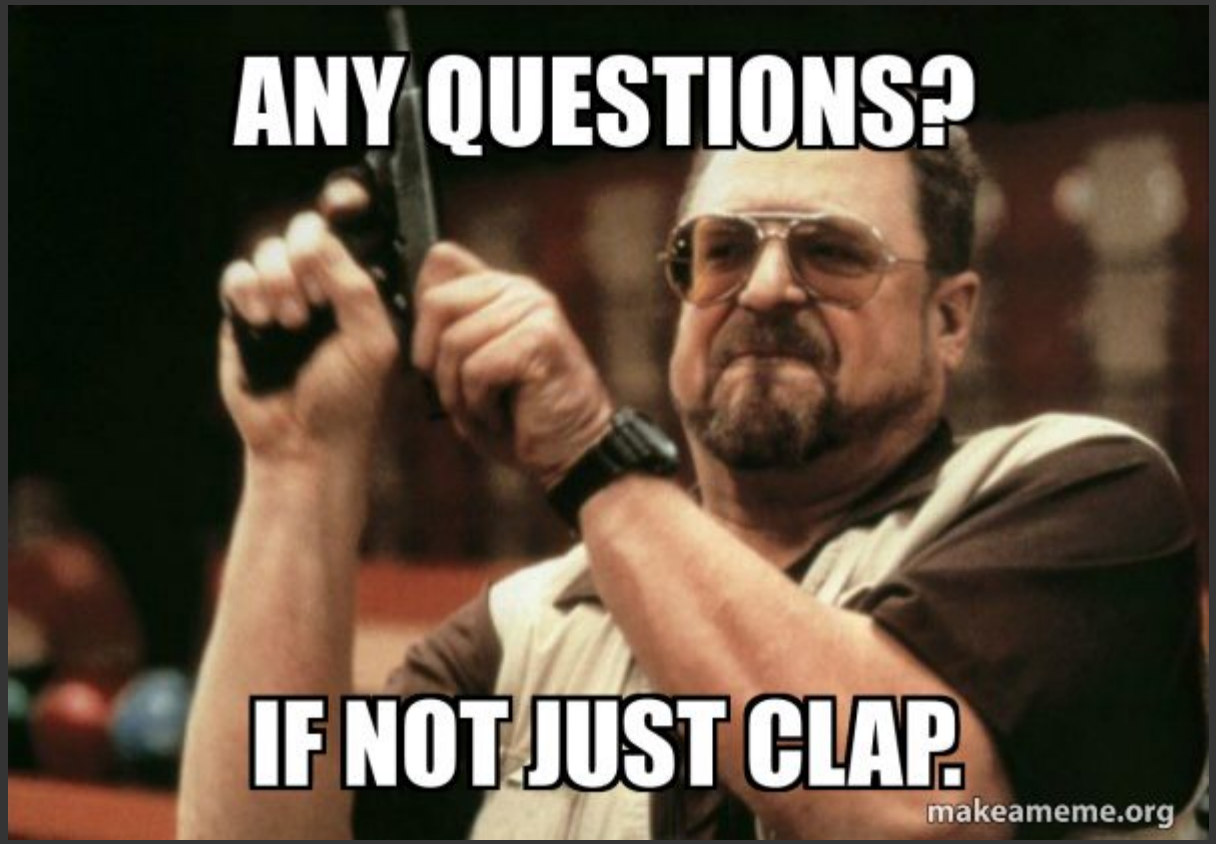

```
T* next() {
    return &internalArray[index++];
}
```

Example 8


```
while (true) {
    std::cout << it->next();
}
```

Example 9

```
class MyVectorIterator {
    std::vector<int>& vec;
    size_t pos = 0;

public:
    MyVectorIterator(std::vector<int>& v) : vec(v) {}

    bool hasNext() {
        return pos < vec.size();
    }

    int next() {
        return vec[pos++];
    }
};
```


ANY QUESTIONS?
IF NOT JUST CLAP.
makeameme.org